CCA 1 / CASE STUDY-BASED PROBLEM SOLVING

Micro-Expression Recognition

A comparison of five models, including an MLP and a standard CNN

Pattern Recognition | Group submission

Member	Name	PRN
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____

STUDY AT A GLANCE

145 labeled clips • 24 subjects • five models • 24 subject-held-out folds

Best macro F1 in this experiment: 49.52% with ResNet-18 and a trained classification head. Results are exploratory and do not establish real-world reliability.

Course / section: _____ Submission date: _____

Activity 1 | Case study selection

Selected problem

Classify brief, subtle facial movements in an already segmented video clip into the three labels used by a research dataset: negative, positive and surprise. The case study is motivated by facial-behavior analysis for human-computer interaction research [1, 2].

Micro-expressions can be difficult to study because their movements are brief and subtle. This study asks how five classifiers compare when evaluated on the same published motion descriptors and on people excluded from their training fold.

Data and scope

The experiment uses the CASME II portion of the MEGC 2019 three-class subset: 145 clips from 24 subjects. The source labels are grouped as negative (disgust or repression), positive (happiness), and surprise; other original categories are excluded [3]. These 145 clips are a selected subset, not the entire CASME II database.

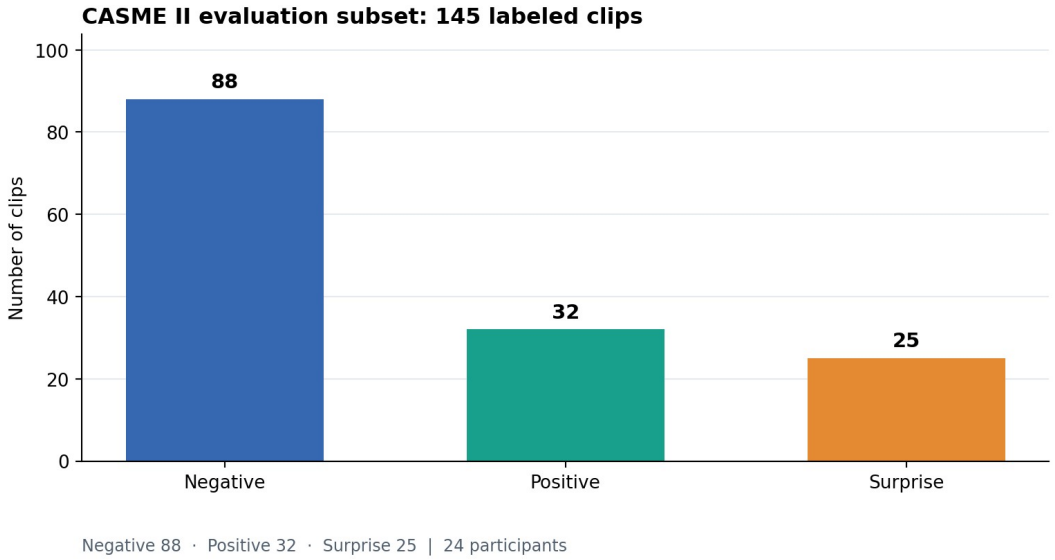


Figure 1. Class distribution of the 145 clips used in the experiment.

The motion descriptors were extracted by the authors of the referenced ATNet repository. Our implementation reads those descriptors; it does not process raw videos or locate micro-expressions in long recordings [2].

Activity 2 | Solution approaches

All approaches use the same 72-value descriptor for each clip. They differ in how they learn the relationship between those descriptors and the three dataset labels.

Approach	How it classifies	Configuration
1. 5-nearest neighbors	Finds nearby training vectors and uses the most common class among them.	k = 5; uniform voting; training-fold feature scaling.
2. RBF support vector machine	Learns a nonlinear decision boundary in the motion-feature space.	C = 1; RBF kernel; gamma = scale; balanced class weights; training-fold scaling.
3. Random forest	Combines the votes of an ensemble of decision trees.	200 trees; balanced class weights; seed 42.
4. MLP neural network	Learns a nonlinear mapping from the 72 input values to three labels.	100-unit hidden layer; Adam; seed 42; fold-specific scaling.
5. ResNet-18 CNN	A standard pretrained CNN processes a three-channel representation of the feature matrix.	ImageNet weights; frozen backbone; new 3-class linear head.

Why compare these models?

- The nearest-neighbor model is an intuitive similarity baseline.
- The SVM can represent nonlinear class boundaries and gives extra weight to minority classes during training.
- The forest offers a tree-ensemble approach that can model feature interactions.
- The MLP and ResNet-18 provide two neural approaches; the ResNet-18 uses pretrained image features.

A most-frequent-class predictor is included as an additional reference baseline. All models use the same clip identities and motion features, although their input transformations and training methods differ.

Metrics

Accuracy is the fraction of clips assigned the correct label. Balanced accuracy averages recall across the three classes. Macro F1 averages the F1 score for each class equally. These class-balanced measures matter because negative clips make up 88 of the 145 samples.

For a class, recall = true positives / (true positives + false negatives), and F1 = 2 × precision × recall / (precision + recall).

Activity 2 | Neural model design

Multilayer perceptron (MLP)

The MLP receives each 8 × 9 HOOF matrix flattened into 72 values. A StandardScaler is fitted on the training subjects only. The scikit-learn MLPClassifier uses one hidden layer with 100 ReLU units, an Adam optimizer, random seed 42 and up to 500 training iterations. It predicts the three class labels.

Input	Training-fold scaling	Neural network	Prediction
72 HOOF values	StandardScaler	100 hidden units	3 class labels

Standard ResNet-18 CNN

The CNN uses the torchvision ResNet-18 architecture with its standard ImageNet-pretrained weights and official preprocessing [4, 5]. The backbone remains frozen. A new three-class linear output head is trained separately for each subject-held-out fold.

The available input is an 8 × 9 motion histogram, not a facial image. The matrix is repeated into three identical channels and fed to the official ImageNet transform: resize to 256 on the short edge, center crop to 224 × 224, then normalize. The pretrained network maps this input to 512 features; the new head produces three logits. The center crop removes some information at the matrix edges.

Input	Standard transform	Frozen CNN	Trainable head
8 × 9 HOOF, repeated to 3 channels	Resize, center crop, ImageNet normalization	ResNet-18; 512 features	Linear: 512 to 3

The linear head uses class-weighted cross-entropy and Adam (learning rate 0.001, weight decay 0.0001) for 200 full-batch epochs. These are documented experiment settings, not official defaults. Applying ImageNet features to a small histogram is exploratory and does not reproduce a face-image CNN or MADCNN.

Activity 3 | Implementation

Inputs and reproducibility

The public research repository supplies one HOOF_TIM10_8 matrix per clip. Each 8 × 9 matrix is flattened to 72 numeric values. The label file is joined to each feature file by clip identifier, rather than assuming matching file order. The loader checks for missing or duplicate IDs, invalid values and unexpected dimensions.

Item	Implementation
Dataset	145 CASME II clips; 24 subject IDs; three classes.
Evaluation	Leave one subject out; 24 folds; every clip receives one held-out prediction.
Scaling	StandardScaler is fitted only on the training portion of each fold for k-NN, SVM and MLP.
Software	Python 3.12.14, NumPy 2.3.5, scikit-learn 1.8.0, PyTorch 2.8.0 and torchvision 0.23.0.
Results	Pooled held-out predictions, metrics, class-level recall and confusion matrices.

Evaluation procedure

1. Load the original feature archive and label file; select and verify CASME II samples.

2. Hold out all clips from one subject. Train each model on the remaining subjects.

3. Scale the training data where required and apply that training-fitted scaler to the held-out data.

4. Predict the held-out subject; repeat until each of the 24 subjects has been held out once.

5. Pool the 145 out-of-fold predictions and calculate the three metrics and confusion matrices.

The script asserts that the training and test subject sets do not overlap and that every sample is tested once. The pretrained CNN is a fixed feature extractor, so only its output head is fitted on training subjects. Settings were fixed for this run; no hyperparameter search was performed.

Activity 4 | Graphical results

The following scores are calculated from pooled predictions for all 145 held-out clips. Balanced accuracy and macro F1 weight the three classes equally; ordinary accuracy weights each clip equally.

Model	Accuracy	Balanced accuracy	Macro F1
5-nearest neighbors	59.31%	36.47%	33.96%
RBF support vector machine	48.97%	40.79%	40.23%
Random forest	58.62%	33.15%	27.04%
Multilayer perceptron	50.34%	36.03%	36.21%
ResNet-18 + linear head	55.17%	51.04%	49.52%
Majority-class baseline	60.69%	33.33%	25.18%

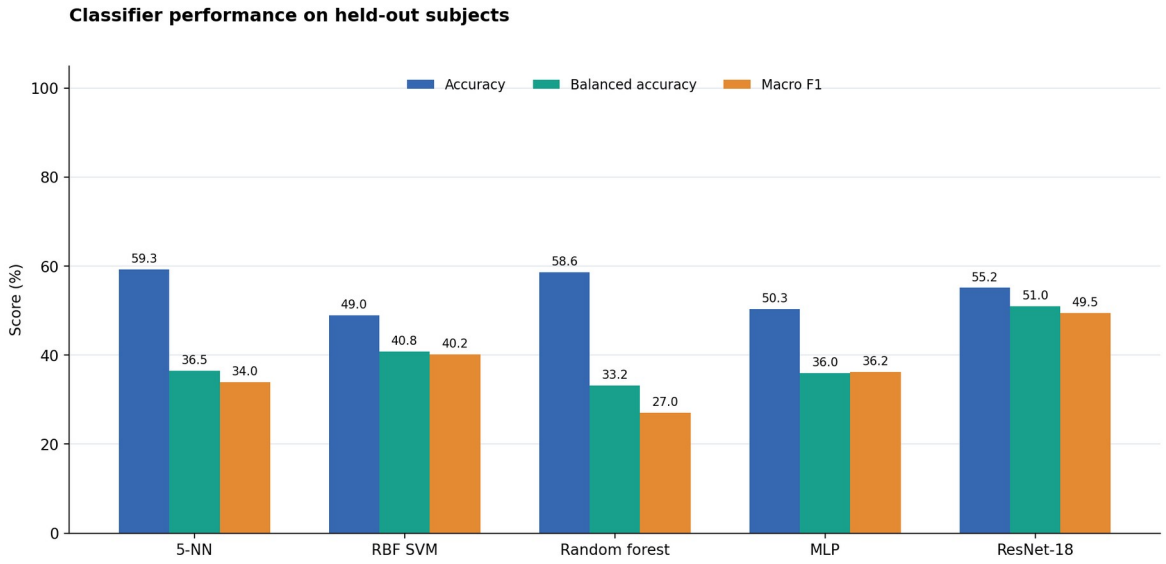


Figure 2. Comparison of five implemented models. The majority baseline is in the table.

Reading the comparison

ResNet-18 and its fitted head have the highest balanced accuracy (51.04%) and macro F1 (49.52%). Their ordinary accuracy is 55.17%. The majority-class baseline reaches 60.69% accuracy by predicting the negative class for every clip, but its macro F1 is only 25.18%.

This small experiment favors the ResNet-18 setup on class-balanced metrics. Different transformations, pretraining and class weighting prevent attributing the difference to architecture alone.

Activity 4 | Class-level results

Rows are the dataset labels and columns are the model predictions. Cell counts are followed by the percentage of that true class; each row sums to approximately 100%.

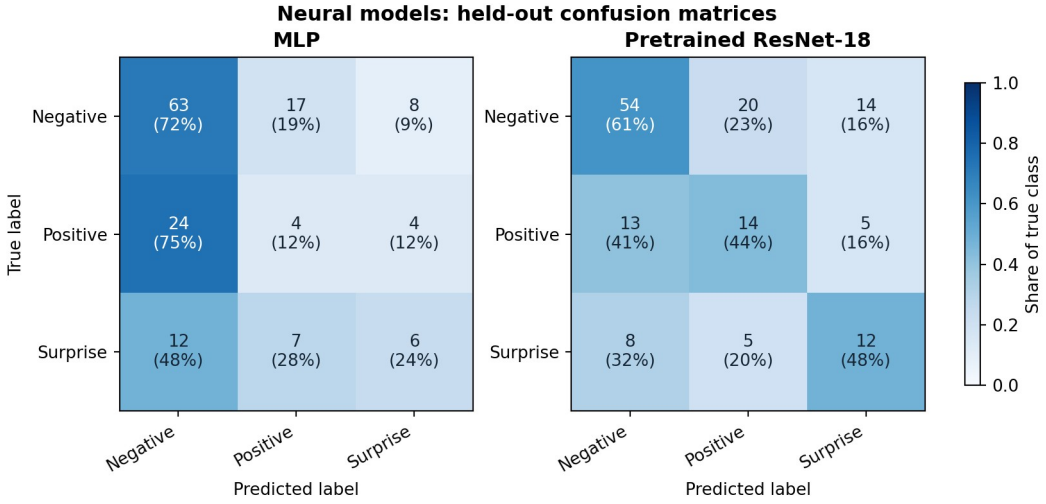


Figure 3. Pooled out-of-fold confusion matrices for the two neural approaches.

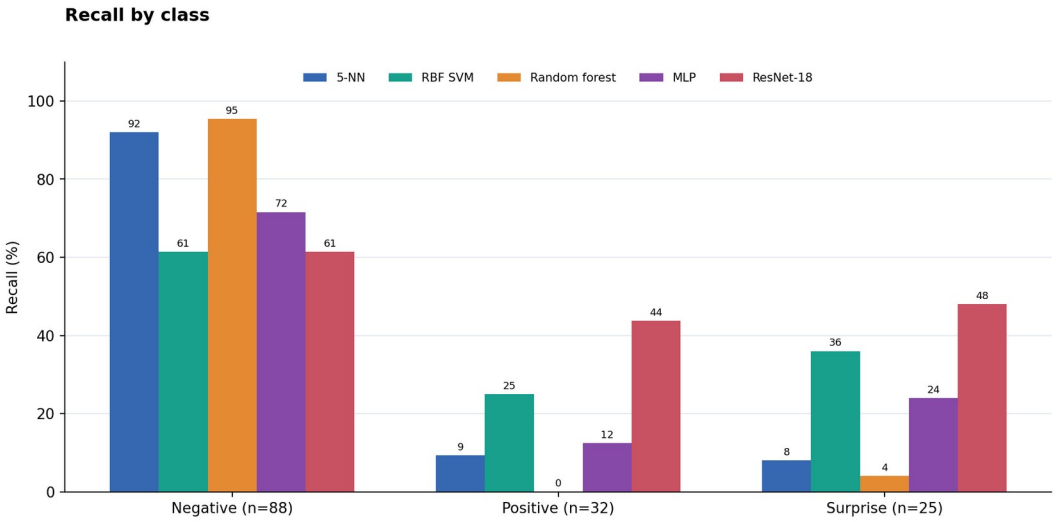


Figure 4. Percentage of clips correctly recognized within each class.

For example, ResNet-18 and its head recognize 54 of 88 negative clips, 14 of 32 positive clips, and 12 of 25 surprise clips. The MLP recognizes 63, 4 and 6 clips in the corresponding classes.

Discussion, limitations and CCA 2

What the results show

- The k-NN and random forest models favor the larger negative class. The ResNet-18 setup obtains higher recall for both minority classes than the other tested configurations.
- The majority baseline illustrates why an imbalanced dataset can make ordinary accuracy look better than minority-class performance.
- The tested configurations differ in input processing, ImageNet pretraining and class weighting. The comparison does not isolate architecture as the sole reason for the differences. The neural models were added after the earlier baseline results were observed.

Limitations

- The dataset is small and laboratory-collected. Results are pooled across clips, so participants with more clips contribute more; they are not an equally weighted average across subjects.
- The classifiers use one published feature representation and fixed settings. The CNN processes enlarged histograms, not face images; the ImageNet transform also crops histogram edges. No external dataset, subject-level confidence interval or nested model-selection procedure was used.
- The system classifies previously segmented and labeled samples. It was not evaluated for live-video spotting, timing, deployment speed, demographic fairness or cross-dataset generalization.
- Dataset labels are annotations, not proof of a person's inner emotional state or truthfulness. Inferring mental states from facial movements has important scientific limitations [6].

How to extend this for CCA 2

Use this case study and the measured results as the starting point for the research paper. Develop a literature review and a testable research question. With appropriate data access, evaluate a standard CNN on actual facial frames or optical-flow images, keep model selection within subject-disjoint training data, and report the new experiment separately.

References and reproducibility

[1] W. J. Yan et al., “CASME II: An Improved Spontaneous Micro-Expression Database and the Baseline Evaluation,” PLOS ONE, 2014. https://doi.org/10.1371/journal.pone.0086041

[2] M. Peng et al., “A Novel Apex-Time Network for Cross-Dataset Micro-Expression Recognition,” ACII, 2019. https://doi.org/10.1109/ACII.2019.8925525. Published feature repository: https://github.com/Trunpm/ACII19-Apex-Time-Network

[3] MEGC 2019, Recognition Challenge guideline: three-class mapping, sample counts and subject-held-out protocol. https://facial-micro-expressiongc.github.io/MEGC2019/images/MEGC2019%20Recognition%20Challenge.pdf

[4] PyTorch, “resnet18 - Torchvision Models,” default ImageNet weights and inference transforms. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

[5] K. He et al., “Deep Residual Learning for Image Recognition,” CVPR, 2016. https://doi.org/10.1109/CVPR.2016.90

[6] L. F. Barrett et al., “Emotional Expressions Reconsidered: Challenges to Inferring Emotion From Human Facial Movements,” Psychological Science in the Public Interest, 2019. https://doi.org/10.1177/1529100619832930

[7] R. Chaudhry et al., “Histograms of Oriented Optical Flow and Binet-Cauchy Kernels on Nonlinear Dynamical Systems for the Recognition of Human Actions,” CVPR Workshops, 2009. https://www.vision.jhu.edu/assets/ChaudhryCVPR09.pdf

[8] scikit-learn, “Cross-validation: evaluating estimator performance,” LeaveOneGroupOut. https://scikit-learn.org/stable/modules/cross_validation.html

Running the code

Extract the project ZIP, create a Python 3.12 environment and install the pinned dependencies as shown in the README. Run python src/run_experiment.py --download. The script downloads and verifies the published features and obtains the official ResNet-18 weights on the first run.

Data access: the repository provides precomputed features, but it does not specify a separate license for them. The original CASME II data has an application process. Check the relevant terms before redistributing data or publishing further work.

The project ZIP includes the source code, numerical metrics, out-of-fold predictions, figures and short discussion notes. Replace the four group placeholders on the cover before submission.